Unidad 3. Elementos de un programa y control de flujos

Contenido

1. Elementos del programa 2
1.1 Estructura y bloques fundamentales 2
1.2 Identificadores y Palabras reservadas 4
1.3 Variables: declaración, inicialización y utilización 5
1.4 Tipos de datos 7
1.5 Literales y Constantes 9
1.6 Operadores y expresiones y Precedencia 11
1.7 Conversiones de tipo: implícitas y explícitas (Casting) 14
1.8 Comentarios 16
2 Control del flujo 18
2.1 Estructuras de selección (Condicionales) 18
2.2 Estructuras de repetición (Bucles) 20
2.3 Estructuras de salto 23
Resumen 25

1. Elementos del programa

Un programa en Java no es un texto plano continuo; es una estructura jerárquica muy organizada. Para que el compilador entienda lo que quieres hacer, debes usar los elementos básicos de construcción correctamente.

Podemos ver el código como una composición de niveles:

1. **Caracteres:** Las letras y símbolos básicos.
2. **Tokens (Léxico):** Las palabras formadas por esos caracteres (palabras clave, nombres).
3. **Sentencias (Sintaxis):** Las frases completas que forman una instrucción.
4. **Bloques:** Los párrafos que agrupan varias sentencias.

Vamos a empezar analizando el esqueleto que sostiene todo esto.

1.1 Estructura y bloques fundamentales

Todo programa en Java, por pequeño que sea, debe respetar una estructura de "muñecas rusas" (una cosa dentro de otra).

A. La Anatomía Básica

El orden obligatorio de contención es: **Paquete (Opcional)** -> **Clase (Obligatorio)** -> **Método (Obligatorio)** -> **Sentencias**

Veamos un ejemplo mínimo y analicémoslo:

Java
```java
package com.miempresa.app;       // 1. PAQUETE

public class HolaMundo {         // 2. CLASE

    public static void main(String[] args) {  // 3. MÉTODO MAIN

        System.out.println("Hola");  // 4. SENTENCIA

    } // Fin del método

} // Fin de la clase
```

1. **Paquete (`package`):** Es la primera línea (si existe). Indica en qué "carpeta" o directorio vive este archivo. Sirve para organizar el código.
2. **Clase (`class`):** Como vimos en la unidad anterior, todo código Java debe vivir dentro de una clase. El archivo debe llamarse igual que la clase (`HolaMundo.java`).
3. **Método `main`:** Es el **Punto de Entrada**. Cuando ejecutas un programa, la Máquina Virtual (JVM) busca este método específico para empezar. Si no lo tienes, el programa no arranca.
4. **Sentencias:** Son las órdenes individuales.

B. Bloques de Código y Llaves `{ }`

Las llaves son los delimitadores que definen dónde empieza y dónde termina una estructura.

- Todo lo que está entre `{` y `}` forma un **Bloque**.
- Los bloques pueden anidarse (un bloque dentro de otro).

El concepto de Ámbito (Scope): Esto es vital. Las variables creadas dentro de un bloque solo existen dentro de ese bloque.

- Si creas una variable dentro del `main`, fuera del `main` (en la clase o en otro método) esa variable no existe. Las llaves definen la "vida" de los datos.

C. El punto y coma `;` (El terminador)

En Java, el salto de línea no significa nada para el ordenador. Puedes escribir todo el programa en una sola línea eterna y funcionaría. Lo que indica el final de una instrucción es el punto y coma (;).

Correcto:

Java
```
int x = 5;
int y = 10;
```

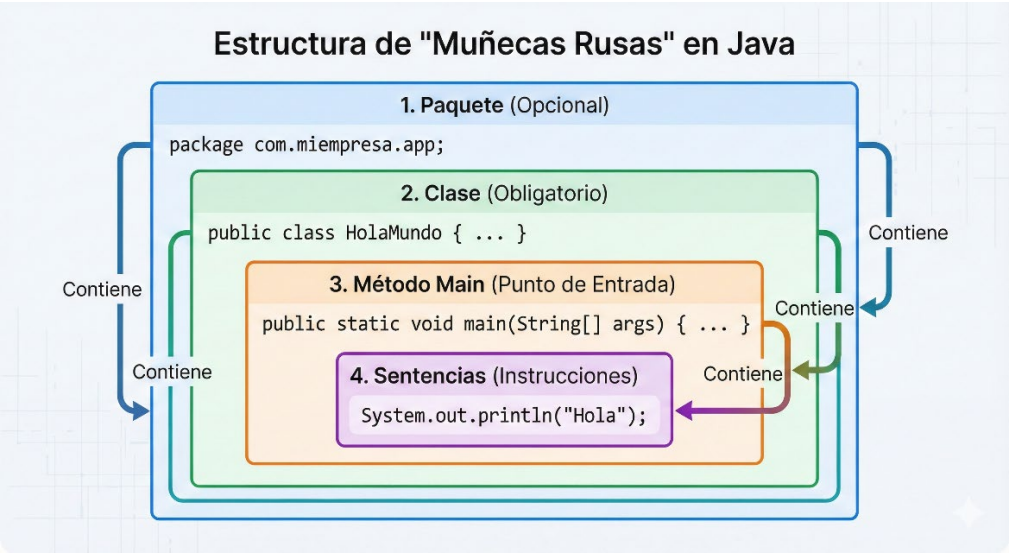
También Correcto (pero ilegible para humanos):

Java
```
int x = 5; int y = 10;
```

Error de compilación:

Java
```
int x = 5
```

(Falta el punto y coma. Es el error más común del principiante).



1.2 Identificadores y Palabras reservadas

A. Identificadores

Un **identificador** es el nombre que tú, como programador, eliges para bautizar a los elementos de tu código: el nombre de una variable, de una clase, de un método o de un paquete.

1. Reglas estrictas (Sintaxis) Si no cumples esto, el programa no compilará:

- **Caracteres permitidos:** Pueden contener letras (A-Z, a-z), números (0-9), el guion bajo (_) y el símbolo de dólar (`$`).
- **Inicio prohibido: Nunca** pueden empezar por un número.
  - `jugador1` (Correcto)
  - `1jugador` (Error)
- **Sin espacios:** No pueden contener espacios en blanco.
- **Case Sensitive:** Java distingue mayúsculas de minúsculas.
  - `nombre`, `Nombre` y `NOMBRE` son tres identificadores totalmente distintos.

2. Convenciones (Estilo Profesional) El código compila si no sigues esto, pero se considera "mal escrito" por la comunidad Java:

- **Clases:** Usan **PascalCase** (Primera letra de cada palabra en mayúscula).
  - Ej: `Coche`, `CuentaBancaria`, `SistemaDeGestion`.
- **Métodos y Variables:** Usan **camelCase** (Primera letra minúscula, siguientes palabras con mayúscula).
  - Ej: `edad`, `calcularSalario`, `nombreDeUsuario`.
- **Constantes:** Usan **SCREAMING_SNAKE_CASE** (Todo mayúsculas separado por guiones bajos).
  - Ej: `PI`, `MAX_VELOCIDAD`.

Identificador	**¿Es válido?**	**¿Es buena práctica?**	**Comentario**
`usuario`	✅ Sí	✅ Sí	Estándar para variables.
`Usuario`	✅ Sí	⚠ No para variables	Parece una Clase.
`2usuarios`	❌ No	-	Empieza por número.

Identificador	¿Es válido?	¿Es buena práctica?	Comentario
`año`	✅ Sí	⚠ Evitar	Evita la 'ñ' o tildes por compatibilidad.
`nombre usuario`	❌ No	-	Tiene un espacio.

B. Palabras Reservadas (Keywords)

Son palabras que el lenguaje Java se ha "reservado" para su propio uso interno. Tienen un significado gramatical especial y **no puedes usarlas como identificadores**.

Por ejemplo, no puedes llamar a una variable `class`, porque Java se confundiría pensando que vas a definir una clase.

Lista común de palabras reservadas (no hace falta memorizarlas todas hoy):

- **Tipos de datos:** `int`, `boolean`, `char`, `void`, `double`.
- **Control de flujo:** `if`, `else`, `switch`, `while`, `for`, `break`, `return`.
- **Modificadores:** `public`, `private`, `static`, `final`, `abstract`.
- **Estructura:** `class`, `package`, `import`, `extends`, `new`, `this`.
- **Valores literales reservados:** `true`, `false`, `null`.

Ejemplo de error:
Java
```
int public = 5; // ¡ERROR! 'public' es una palabra reservada.
int Public = 5; // Válido (la P mayúscula la diferencia), pero mala
práctica.
```

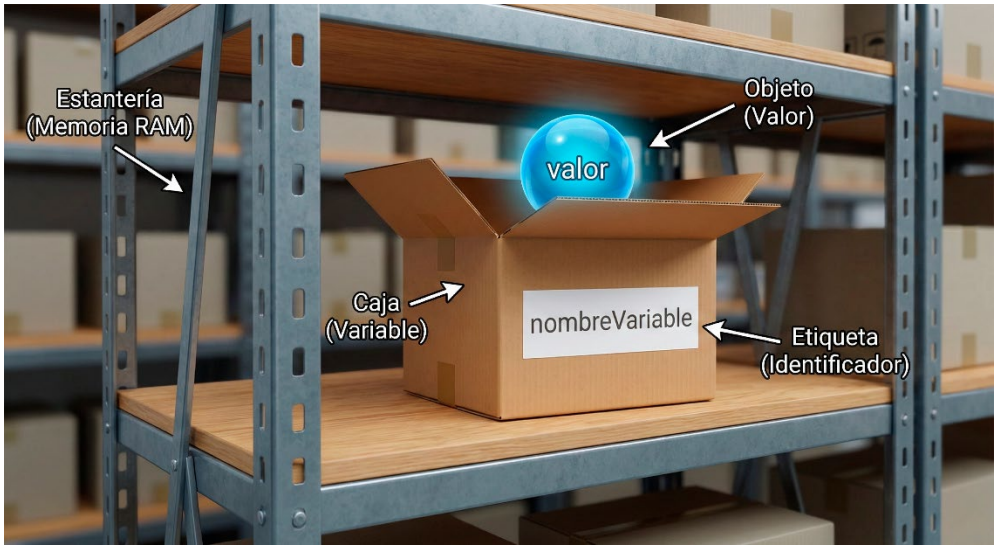
1.3 Variables: declaración, inicialización y utilización

Una **variable** es un espacio reservado en la memoria del ordenador para almacenar un dato que puede cambiar (variar) durante la ejecución del programa.

La Analogía de la Caja

Imagina que la memoria RAM es una estantería gigante.

- Una **variable** es una **caja** que pones en esa estantería.
- La caja tiene una **etiqueta** (Identificador/Nombre) para que la encuentres.
- La caja tiene una **forma específica** (Tipo de dato) que determina qué puedes guardar dentro (zapatos, libros, agua...). No puedes guardar agua en una caja de cartón para libros.



A. Ciclo de Vida de una Variable

En Java, para usar una variable, debes seguir estrictamente tres pasos. Java es un lenguaje de **tipado fuerte**, lo que significa que debes avisar de antemano qué tipo de dato vas a guardar.

1. Declaración (Crear la caja). Le dices al sistema: "Resérvame espacio para un dato de tipo X y llámalo Y".

- Sintaxis: `TipoDeDato nombreVariable;`

Java
```
int edad;        // Declaro una variable para números enteros
String nombre;   // Declaro una variable para texto
```

2. Inicialización (Meter algo en la caja). Es la primera vez que le asignas un valor. Hasta que no haces esto, la caja está "basura" o vacía, y Java no te dejará usarla.

- Sintaxis: `nombreVariable = valor;`
- El operador = es de **asignación** (significa "guarda esto ahí"), no de igualdad matemática.

Java
```
edad = 25;           // Guardo el 25 en 'edad'
nombre = "Carlos";   // Guardo "Carlos" en 'nombre'
```

3. Utilización (Usar el contenido). Llamas a la variable por su nombre para leer lo que tiene dentro o modificarlo.

Java
```
System.out.println(edad); // Imprime 25
edad = 26;                // Cambio el valor (el 25 se pierde para
siempre)
```

Forma Abreviada (La más común): Puedes declarar e inicializar en una sola línea.

Java
```
int edad = 25;
```

B. El error más común: Variable no inicializada

Java es muy protector. Si intentas usar una variable que has declarado, pero no has inicializado (está vacía), el compilador te dará un error y no te dejará ejecutar el programa.

Java
```
int puntuacion;
// System.out.println(puntuacion); // ¡ERROR DE COMPILACIÓN!
// Java dice: "Oye, la caja está vacía, no puedo imprimir nada".
```

C. Almacenamiento en Memoria (¿Dónde viven?)

Dependiendo de dónde declares la variable, se guarda en un sitio distinto de la RAM. Esto es crucial para entender el rendimiento y el comportamiento.

1. **Variables Locales (En el Stack/Pila):**
   - Son las que declaras **dentro de un método** (como en el `main`).
   - **Vida:** Nacen cuando empieza el método y mueren (se borran de la RAM) en cuanto se cierra la llave `}` del método.
   - **Almacenamiento:** Se guardan en el **Stack**. Es una memoria muy rápida y pequeña. Guardan directamente el valor (si es primitivo) o la referencia (si es objeto).
2. **Variables de Instancia (Atributos):**
   - Son las que declaras dentro de la Clase, pero fuera de los métodos.
   - **Vida:** Viven tanto tiempo como viva el objeto al que pertenecen.
   - **Almacenamiento:** Se guardan en el **Heap** (dentro del objeto).

1.4 Tipos de datos

A. Tipos Primitivos (Los átomos)

Son los tipos básicos que vienen integrados en el lenguaje. No son objetos (no tienen métodos, no puedes hacer `numero.algo()`). Almacenan el **valor real** directamente en la memoria Stack. Son muy rápidos y ligeros.

Existen exactamente **8 tipos primitivos** en Java, divididos en 4 grupos:

1. Números Enteros (Sin decimales)

Se diferencian por el tamaño de memoria que ocupan y el rango de números que pueden guardar.

- **`byte`** (8 bits): Muy pequeño. De -128 a 127. Útil para ahorrar memoria en arrays gigantes o procesar archivos binarios.
- **`short`** (16 bits): Pequeño. De -32,768 a 32,767. Poco uso hoy en día.

- `int` (32 bits): **El estándar.** Es el que usarás el 99% de las veces para contar. Acepta hasta ±2 mil millones.
- `long` (64 bits): Para números gigantescos (como habitantes del planeta o milisegundos).
    - *Ojo:* Se debe poner una **'L'** o **'l'** al final del número: `long poblacion = 7000000000L;`.

2. Números en Coma Flotante (Con decimales)

Para guardar números reales como 3.14 o precios.

- `float` (32 bits): Precisión simple (6-7 dígitos decimales).
    - *Ojo:* Se debe poner una **'F'** o **'f'** al final: `float precio = 19.99f;`.
- `double` (64 bits): **El estándar.** Precisión doble (15 dígitos decimales). Es el que se usa por defecto en Java para matemáticas.

3. Carácter

- `char` (16 bits): Guarda **un solo** carácter Unicode.
    - Se escribe siempre con **comillas simples**: `'A'`, `'ñ'`, `'9'`.
    - Por dentro, Java lo guarda como un número positivo (código Unicode), por lo que se pueden sumar (ej: 'A' + 1 = 'B').

4. Lógico (Booleano)

- `boolean`: Solo puede tener dos valores: `true` (verdadero) o `false` (falso). Fundamental para las condiciones (`if`).

B. Tipos de Referencia (Los Objetos)

Cualquier tipo de dato que **no** sea uno de los 8 primitivos anteriores es un Tipo de Referencia.

- Almacenan una **dirección de memoria** (referencia) que apunta a un objeto en el Heap.
- Pueden tener métodos y propiedades.
- El valor por defecto si no se inicializan es `null`.

El más importante en esta etapa es:

- `String` **(Cadena de texto):**
    - No es primitivo, es una **Clase**.
    - Sirve para guardar texto.
    - Se escribe con **comillas dobles**: `"Hola Mundo"`.

- Tiene métodos poderosos (ej: `.toUpperCase()`, `.length()`).

Diferencia importante:

- `char letra = 'a';` (Comillas simples, primitivo, un solo símbolo).
- `String texto = "a";` (Comillas dobles, objeto, cadena de longitud 1).

C. Tabla Resumen y Literales

Aquí tienes la chuleta definitiva para elegir el tipo de dato:

Tipo	Tamaño	Ejemplo de Uso	Ejemplo de Código
boolean	1 bit (teórico)	Interruptor (ON/OFF)	`boolean encendido = true;`
char	2 bytes	Una letra	`char inicial = 'J';`
int	4 bytes	Contadores, edad, ID	`int edad = 25;`
long	8 bytes	Cifras astronómicas	`long distancia = 15000000000L;`
double	8 bytes	Precios, medidas, física	`double peso = 75.5;`
String	Variable	Nombres, frases	`String nombre = "Juan";`

Importante: Ante la duda, usa `int` para enteros y `double` para decimales. Son los tipos por defecto que Java asume cuando escribes un número.

1.5 Literales y Constantes

A. Literales

Un **literal** es cualquier valor constante escrito directamente ("a mano") en el código fuente. Es la representación textual de un dato.

Cuando escribes int x = 5;, el 5 es un literal entero.

Cuando escribes String s = "Hola";, el "Hola" es un literal de cadena.

Es importante saber escribirlos bien, porque Java infiere el tipo de dato según cómo lo escribas.

1. Literales Numéricos Enteros

- **Decimal:** El estándar. Ej: `100`, `-5`.
- **Long:** Si el número es muy grande, debes añadir **`L`** o `l`. Ej: `9000000000L`.
- **Binario/Hexadecimal:** Java permite escribir en otras bases (útil en sistemas).
    - Binario (prefijo `0b`): `0b101` (es el 5).
    - Hexadecimal (prefijo `0x`): `0xFF` (es el 255).
- **Truco de legibilidad:** Puedes usar guiones bajos _ para separar miles. Java los ignora.
    - `int millon = 1_000_000;` (Mucho más fácil de leer).

2. Literales Reales (Decimales)

- **Double:** Por defecto, cualquier número con punto es double. Ej: `3.14`.
- **Float:** Debes forzarlo con **`F`** o `f`. Ej: `3.14F`. Si olvidas la F, dará error de compilación.

3. Literales de Carácter y Texto. La diferencia es las comillas.

- **`char`:** Comillas **simples**. Solo un carácter. Ej: `'A'`, `'9'`, `'%'`.
    - **Secuencias de escape:** Caracteres especiales que empiezan por `\`.
        - `'\n'` (Salto de línea).
        - `'\t'` (Tabulador).
        - `'\''` (Comilla simple dentro de un char).
- **`String`:** Comillas **dobles**. Texto de cualquier longitud. Ej: `"Hola mundo"`.

4. Literales Booleanos. Solo existen dos: `true` y `false`. No se usan comillas (si pones "`true`", es un String).

B. Constantes (`final`)

Una constante es una variable cuyo valor **no puede cambiar** una vez inicializada. Es una caja que, tras meter el dato, la sellamos con pegamento fuerte.

En Java, usamos la palabra reservada **`final`**.

Sintaxis:
Java
```
final TipoDeDato NOMBRE_CONSTANTE = valor;
```

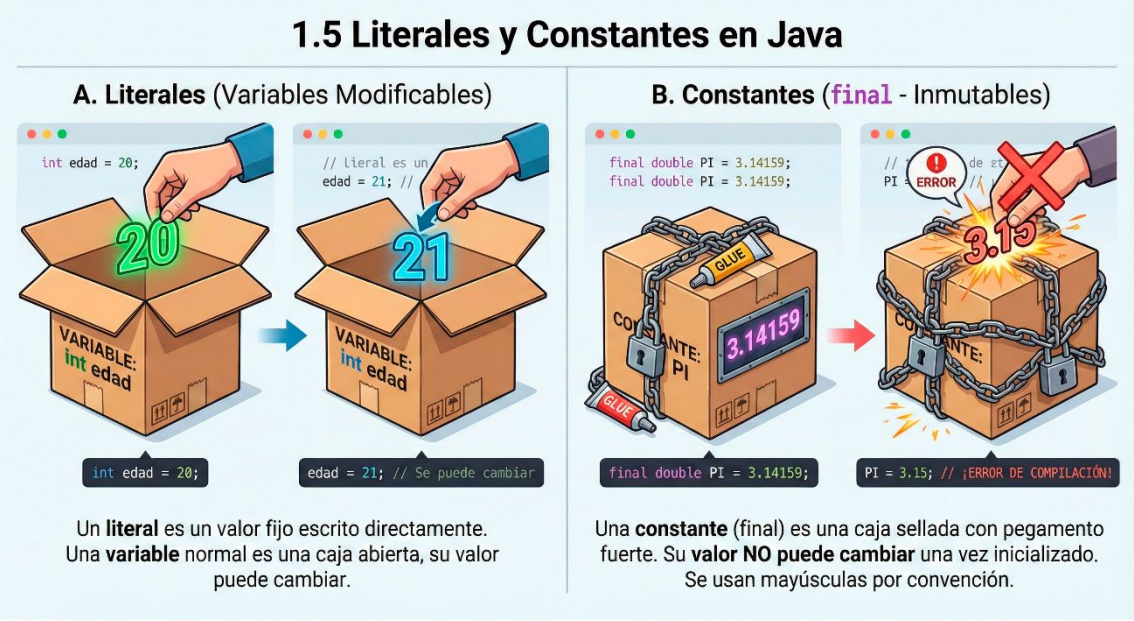
Reglas de Estilo: Por convención mundial, las constantes se escriben **TODO_EN_MAYUSCULAS**, separando las palabras con guiones bajos (_). Esto permite identificar de un vistazo qué valores son fijos.

Ejemplo:
Java
```
public class Matematicas {
    public static void main(String[] args) {
        // Variable normal (puede cambiar)
```

```
        int edad = 20;
        edad = 21; // ✅ Correcto

        // Constante (no puede cambiar)
        final double PI = 3.14159;
        final int MAX_USUARIOS = 100;

        // PI = 3.15; // ❌ ERROR DE COMPILACIÓN: No se puede
asignar un valor a una variable final
    }
}
```



¿Por qué usar constantes? (El problema de los "Magic Numbers")

Imagina que lees este código:

Java
```
double precioFinal = precio * 1.21;
```

¿Qué es `1.21`? Es un "número mágico". Un número suelto que nadie sabe de dónde sale. Si el IVA cambia mañana, tienes que buscar todos los `1.21` de tu código y rezar para no equivocarte.

La forma profesional es usar constantes:

Java
```
final double IVA_ESPAÑA = 1.21;
double precioFinal = precio * IVA_ESPAÑA;
```

Ahora el código se explica solo (autodocumentado) y es fácil de mantener.

1.6 Operadores y expresiones y Precedencia

A. Conceptos Básicos

- **Operador:** Un símbolo que realiza una acción sobre uno o más valores (operandos). Ej: `+`, `-`, `*`, `==`.
- **Expresión:** Una combinación de variables, literales y operadores que, al ser evaluada por Java, produce un **resultado único**.
    - `5 + 3` es una expresión que devuelve `8`.
    - `edad > 18` es una expresión que devuelve `true` o `false`.

B. Tipos de Operadores

Clasificamos los operadores según qué hacen con los datos.

1. Operadores Aritméticos (Matemáticas)

Sirven para calcular números.

- **+** (Suma): `10 + 5` → `15`
- **-** (Resta): `10 - 5` → `5`
- ***** (Multiplicación): `10 * 5` → `50`
- **/** (División): **¡CUIDADO AQUÍ!**
    - Si divides dos enteros, el resultado es **entero** (se trunca el decimal).
    - `5 / 2` → **2** (No da 2.5).
    - Para obtener decimales, uno de los números debe ser real: `5.0 / 2` → `2.5`.
- **%** (Módulo o Resto): Devuelve el resto de la división entera.
    - `10 % 3` → `1` (Porque 10 entre 3 es 3 y sobra 1).
    - *Uso típico:* Saber si un número es par (`n % 2 == 0`) o impar.

2. Operadores de Asignación

Sirven para guardar valores en variables. Siempre funcionan de **derecha a izquierda**.

- **=** (Asignación simple): `x = 5;` (Guarda el 5 en x).
- **Asignación compuesta:** Realizan una operación y guardan el resultado de golpe. Ahorran escribir código.
    - `x += 5` equivale a `x = x + 5`
    - `x -= 2` equivale a `x = x - 2`

3. Operadores Relacionales (Comparación)

Comparan dos valores y **siempre devuelven un booleano** (`true` o `false`). Son la base de las condiciones (`if`).

- **==** (Igualdad): `5 == 5` → `true`.
    - ⚠ **Error clásico:** No confundir con `=` (asignación).

- **!=** (Desigualdad/Distinto): `5 != 3` → `true`.
- **>** (Mayor que): `5 > 3` → `true`.
- **<** (Menor que): `5 < 8` → `true`.
- **>=** (Mayor o igual).
- **<=** (Menor o igual).

4. Operadores Lógicos

Sirven para combinar varias condiciones booleanas.

- **&&** (AND / Y): Devuelve `true` solo si **AMBAS** partes son verdaderas.
  - `(edad > 18) && (tieneDinero == true)`
- **||** (OR / O): Devuelve `true` si **AL MENOS UNA** es verdadera.
  - `(esFinDeSemana) || (estoyDeVacaciones)`
- **!** (NOT / Negación): Invierte el valor.
  - `!true` → `false`.

5. Operadores Unarios (Incremento/Decremento)

Modifican una variable numérica sumando o restando 1. Muy usados en bucles `for`.

- **++** (Incrementar): `i++` aumenta el valor de `i` en 1.
- **--** (Decrementar): `i--` reduce el valor de `i` en 1.

Matiz Pro (Prefijo vs Sufijo):

- `++i` (Pre-incremento): Sube el valor **antes** de usar la variable.
- `i++` (Post-incremento): Usa la variable y **luego** sube el valor.

C. Precedencia de Operadores (La Jerarquía)

Si escribes `int x = 5 + 3 * 2;`, ¿cuánto vale x?

- ¿Es `(5+3)*2 = 16`?
- ¿O es `5+(3*2) = 11`?

Java sigue un orden estricto de evaluación (muy parecido a las matemáticas). **Se evalúa de arriba a abajo**:

1. **Paréntesis** `()`: Son los reyes. Todo lo que esté dentro se hace primero. Úsalos para forzar el orden.
2. **Operadores Unarios**: `++`, `--`, `!`
3. **Multiplicativos**: `*`, `/`, `%`

4. **Aditivos**: `+`, `-`
5. **Relacionales**: `>`, `<`, `==`
6. **Lógicos**: `&&`, `||`
7. **Asignación**: `=`, `+=` (Siempre es lo último que ocurre).

En el ejemplo anterior: `5 + 3 * 2` → Java ve el `*` antes que el `+`. Multiplica `3*2` (6) y luego suma `5`. Resultado: **11**.

> **Consejo:** No te fíes de tu memoria con la precedencia. Si tienes una expresión compleja, **usa paréntesis,** aunque no sean obligatorios. Hacen el código más legible y seguro.

- `a + b * c` (Confuso)
- `a + (b * c)` (Claro)

1.7 Conversiones de tipo: implícitas y explícitas (Casting)

A. Conversión Implícita (Automática / Widening)

Ocurre cuando pasamos un valor de una caja pequeña a una caja más grande.

Como hay espacio de sobra, Java realiza la conversión automáticamente sin que tengas que escribir nada extra. No hay riesgo de pérdida de datos.

- Dirección: De menor capacidad a mayor capacidad. `byte` → `short` → `int` → `long` → `float` → `double`

Ejemplo: Imagina verter un vaso de agua (`int`) en un cubo grande (`double`). Cabe perfectamente.

Java
```
int numeroEntero = 100;

// Conversión IMPLÍCITA:
// Java transforma el 100 en 100.0 automáticamente para guardarlo
en el double.
double numeroReal = numeroEntero;

System.out.println(numeroReal); // Imprime: 100.0
```

B. Conversión Explícita (Casting / Narrowing)

Ocurre cuando intentamos pasar un valor de una caja grande a una caja más pequeña.

Aquí hay un problema físico: el dato podría no caber. Java se niega a hacerlo automáticamente para protegerte (error de compilación: "possible loss of precision").

Si estás seguro de querer hacerlo, debes **forzar** la conversión manualmente usando el **Casting**.

- **Sintaxis:** Escribir el tipo destino entre paréntesis delante del valor: `(TipoDestino) valor`.

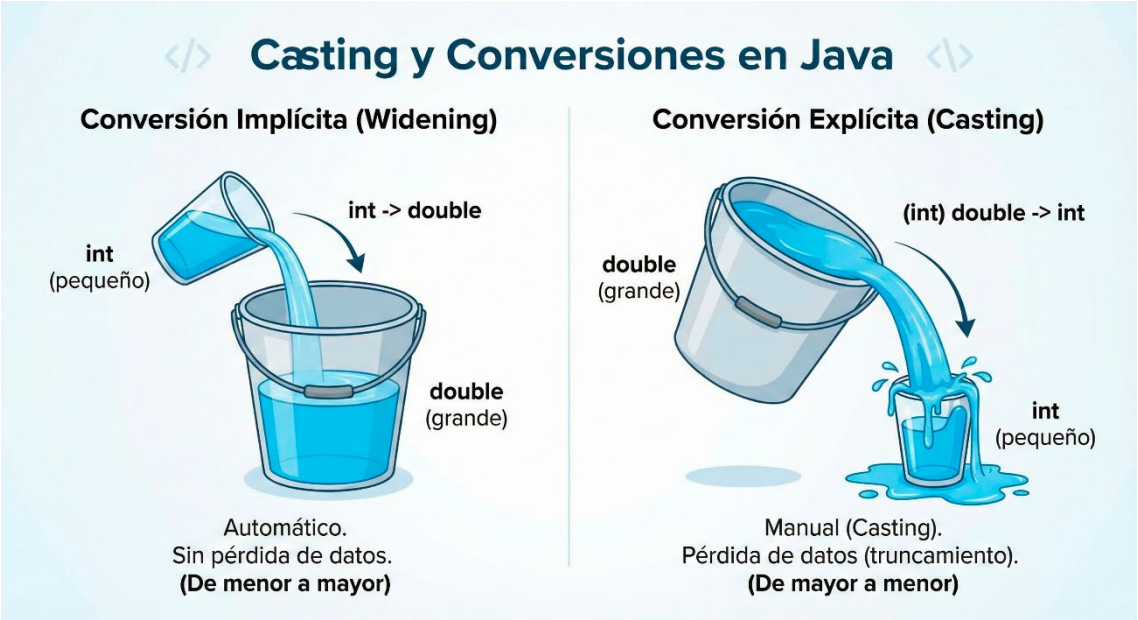
Ejemplo: Imagina querer meter el agua de un cubo (double) en un vaso (int). Es probable que el agua se desborde (pierdas los decimales).

Java
```
double precio = 9.99;

// int precioEntero = precio; // ❌ ERROR DE COMPILACIÓN: Java te protege.

// Conversión EXPLÍCITA (Casting):
// "Sé lo que hago, conviértelo a entero aunque pierda datos".
int precioEntero = (int) precio;

System.out.println(precioEntero); // Imprime: 9
```



¡Importante! Truncamiento, no redondeo. Fíjate en el ejemplo anterior: `9.99` pasa a ser `9`. El casting de decimal a entero trunca (corta los decimales con tijeras), no redondea al más cercano.

C. Tabla Resumen de Riesgos

Tipo de Conversión	De... A...	Sintaxis	¿Peligro?
Implícita	Pequeño → Grande (`int` a `double`)	`a = b;`	✅ Ninguno (Seguro)
Explícita (Cast)	Grande → Pequeño (`double` a `int`)	`a = (int) b;`	⚠ Pérdida de datos (decimales o desbordamiento)

D. Casos Especiales

1. El caso del `char` y el `int`. Los caracteres en el fondo son números (tabla Unicode). Puedes hacer casting entre ellos.

- `char` a `int`: Obtienes el código numérico de la letra (Implícito).
- `int` a `char`: Obtienes la letra asociada a ese código (Explícito, porque `int` es más grande que `char`).

Java
```
char letra = 'A';
int codigo = letra; // Implícito. Código vale 65

int num = 66;
char letra2 = (char) num; // Explícito (Casting). letra2 vale 'B'
```

2. Conversión a `String` (No es casting). Un error muy común de principiante es intentar hacer casting a String: `(String) 5`. **Esto no funciona.** Para convertir números a texto, usamos la concatenación o métodos auxiliares.

Java
```
int numero = 50;
// String texto = (String) numero; // ❌ ERROR FATAL

String texto = String.valueOf(numero); // ✅ Correcto (Método helper)
String texto2 = "" + numero;           // ✅ Correcto (Truco de concatenación)
```

1.8 Comentarios

Un **comentario** es un fragmento de texto dentro del código fuente que el compilador de Java **ignora por completo**. No afecta en absoluto al tamaño ni al rendimiento del programa. Su único destinatario es el ser humano. En Java, disponemos de tres tipos de comentarios:

A. Comentario de una sola línea (`//`)

Es el más utilizado para notas breves. Todo lo que escribas desde las dos barras `//` hasta el final de esa línea será ignorado.

- **Uso:** Explicaciones rápidas o anular una línea de código temporalmente.
  Java
  ```
  // Esto es una variable para la edad
  int edad = 25;

  edad = 30; // Actualizamos la edad
  // System.out.println(edad); -> Esta línea no se ejecutará (está comentada)
  ```

B. Comentario Multilínea o de Bloque (`/* ... */`)

Permite escribir texto que ocupa varias líneas. Empieza con `/*` y termina con `*/`. Todo lo que haya en medio, aunque sean 100 líneas, será ignorado.

- **Uso:** Explicar algoritmos complejos al principio de un archivo o desactivar bloques grandes de código.

  Java
  ```
  /*
   Este bloque calcula el área de un círculo.
   Fórmula: Pi * r^2
   Autor: Gemini
  */
  double area = 3.14 * radio * radio;
  ```

C. Comentarios de Documentación (Javadoc) (`/** ... */`)

Este es especial y exclusivo de Java. Empieza con doble asterisco /** y termina con */. Aunque el compilador lo ignora, existe una herramienta llamada javadoc que lee estos comentarios y genera automáticamente una página web HTML con la documentación de tu proyecto.

- **Uso:** Se coloca **justo antes** de declarar clases o métodos importantes.
- **Etiquetas:** Usa palabras clave como `@param` (para describir parámetros), `@return` (para describir qué devuelve) o `@author`.

  Java
  ```
  /**
   * Calcula la suma de dos números enteros.
   * * @param a El primer número
   * @param b El segundo número
   * @return La suma de a y b
   */
  public int sumar(int a, int b) {
      return a + b;
  }
  ```

D. La filosofía del "Clean Code" (Código Limpio)

Saber poner comentarios es un arte. Aquí tienes la regla de oro:

> **"No comentes el QUÉ, comenta el POR QUÉ".**

El código ya dice *qué* está pasando (si sabes leer Java). El comentario debe explicar la razón de negocio o la decisión compleja detrás.

- ❌ **Mal comentario (Ruido):**

  Java
  ```
  int i = 0; // Declaro i y le asigno 0
  i++; // Incremento i en 1
  ```

  ¿Por qué está mal? Porque es obvio. Solo ensucia la lectura.

- ✅ **Buen comentario (Contexto):**

  Java

```
// Añadimos 10ms de espera porque la base de datos antigua es
lenta
Thread.sleep(10);
```

¿Por qué está bien? Explica una razón que no se ve en el código.

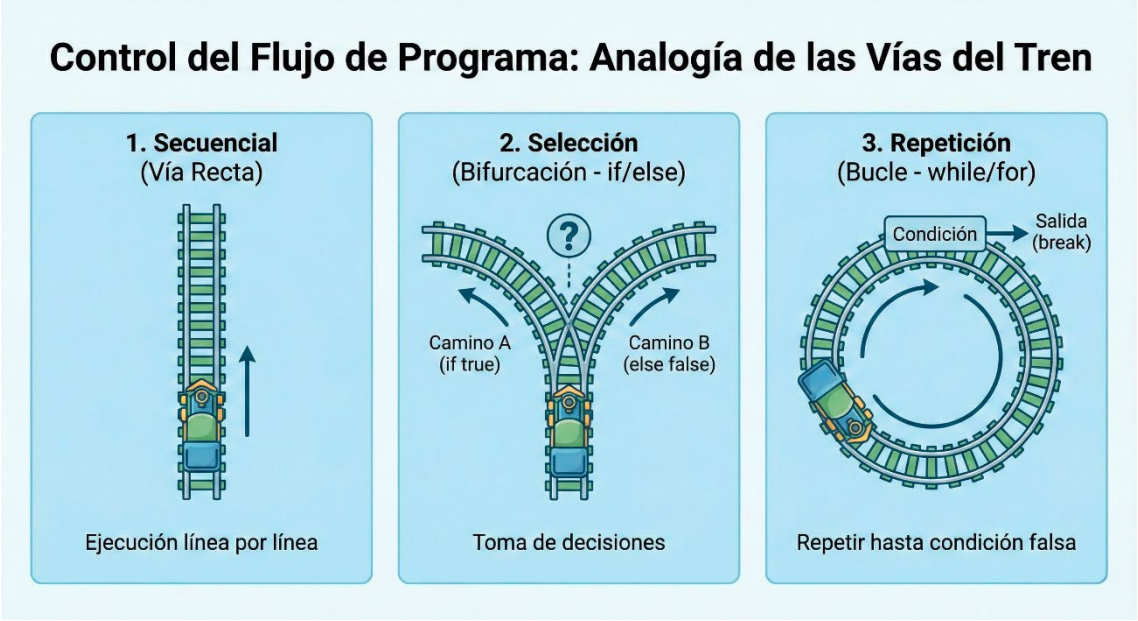
2 Control del flujo

Las estructuras de control son sentencias que rompen la ejecución secuencial del programa, permitiendo que el código salte a otras partes, decida qué ejecutar o repita fragmentos.

Podemos imaginarlo como las vías de un tren:

1. **Secuencial:** Una vía recta.
2. **Selección:** Un cambio de agujas (desvío a la izquierda o derecha).
3. **Repetición:** Un circuito circular (dar vueltas).

Empecemos por la toma de decisiones.



2.1 Estructuras de selección (Condicionales)

Estas estructuras permiten ejecutar un bloque de código **solo si se cumple una condición determinada**. Esa condición siempre debe ser una expresión **booleana** (es decir, debe dar como resultado `true` o `false`).

A. Sentencia `if` (Si...)

Es la estructura más básica. "Si pasa esto, haz esto".

Java
```
if (condicion) {
    // Código que se ejecuta SOLO si la condición es true
}
```

Ejemplo:
Java
```
int nota = 8;

if (nota >= 5) {
    System.out.println("¡Aprobado!"); // Se imprime porque 8 >= 5
es true
}
```

B. Sentencia `if-else` **(Si... Si no...)**

Añade una alternativa. "Si pasa esto, haz A. Si no pasa, haz B". Nunca se ejecutan los dos a la vez; son caminos excluyentes.

Java
```
int nota = 4;

if (nota >= 5) {
    System.out.println("Aprobado");
} else {
    // Esto se ejecuta si la condición de arriba fue false
    System.out.println("Suspenso");
}
```

C. Sentencia `if-else if` **(Escalera de decisiones)**

Cuando tienes más de dos opciones. Java evalúa las condiciones en orden, de arriba a abajo. **En cuanto una se cumple, entra en ella, ejecuta su código y salta al final de toda la estructura**.
Java
```
int hora = 14;

if (hora < 12) {
    System.out.println("Buenos días");
} else if (hora < 20) {
    System.out.println("Buenas tardes"); // Entra aquí
} else {
    System.out.println("Buenas noches");
}
```

D. Operador Ternario (`? :`)

Es una forma "mini" y elegante de escribir un `if-else` en una sola línea. Se usa mucho para asignaciones rápidas.

- **Sintaxis:** `condición ? valor_si_true : valor_si_false;`

Java
```
int edad = 17;
// Si edad >= 18 guarda "Mayor", si no, guarda "Menor"
String estado = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
```

E. Sentencia `switch` **(Interruptor)**

Imagina que quieres crear un menú con opciones (1, 2, 3...). Usar un montón de `else if (opcion == 1)... else if (opcion == 2)`... es sucio y lento. Para eso existe el `switch`. Evalúa una variable y salta directamente al caso (`case`) que coincida con su valor.

> **Limitación:** El `switch` clásico solo funciona para comparar igualdad de valores concretos (no rangos como "mayor que"). Tipos permitidos: `int`, `char`, `String`, `byte`, `short` y `Enum`.

Java
```
int diaSemana = 3;

switch (diaSemana) {
    case 1:
        System.out.println("Lunes");
        break; // IMPORTANTE: El break evita que siga bajando
    case 2:
        System.out.println("Martes");
        break;
    case 3:
        System.out.println("Miércoles"); // Entra aquí
        break;
    case 4:
        System.out.println("Jueves");
        break;
    case 5:
        System.out.println("Viernes");
        break;
    default:
        // Se ejecuta si no coincide con ningún case anterior (como
el 'else')
        System.out.println("Es fin de semana o error");
}
```

El peligro del `break`: En el `switch` clásico, si se te olvida poner el `break`, ocurre el efecto **Fall-Through** (caída libre): Java ejecutará el caso correcto **Y TAMBIÉN** todos los que haya debajo hasta encontrar un break o el final.

- *A veces se usa a propósito:* Por ejemplo, agrupar `case 1: case 2: case 3: System.out.println("Laborable"); break;`.

Importante: Java introdujo una sintaxis mejorada ("Switch Expressions") que usa flechas -> y no necesita break, evitando errores, pero para empezar es mejor dominar la sintaxis clásica mostrada arriba.

2.2 Estructuras de repetición (Bucles)

Un **bucle** (o *loop*) es una estructura que repite un bloque de instrucciones mientras se cumpla una condición determinada (es decir, mientras la condición sea `true`).

En Java existen tres tipos principales, cada uno ideal para una situación distinta.

A. El Bucle `while` (Mientras)

Es el bucle más sencillo. **Piensa antes de actuar.**

1. Evalúa la condición.
2. Si es verdadera, entra y ejecuta el código.
3. Vuelve al paso 1.
4. Si es falsa, salta fuera del bucle.

- **Uso ideal:** Cuando **NO sabes** cuántas veces se va a repetir algo (ej: "Sigue leyendo el archivo hasta que se acabe").

Java
```
int contador = 1;

while (contador <= 5) {
    System.out.println("Vuelta número: " + contador);
    contador++; // IMPORTANTE: Actualizar la variable de control
}
// Salida: Imprime del 1 al 5
```

¡Peligro! El Bucle Infinito

Si olvidas poner contador++, la condición 1 <= 5 será siempre verdadera. El programa nunca terminará y se quedará "colgado" consumiendo CPU hasta que lo fuerces a cerrarse.

B. El Bucle `do-while` (Hacer... Mientras)

Es una variante del anterior. Actúa primero, piensa después.

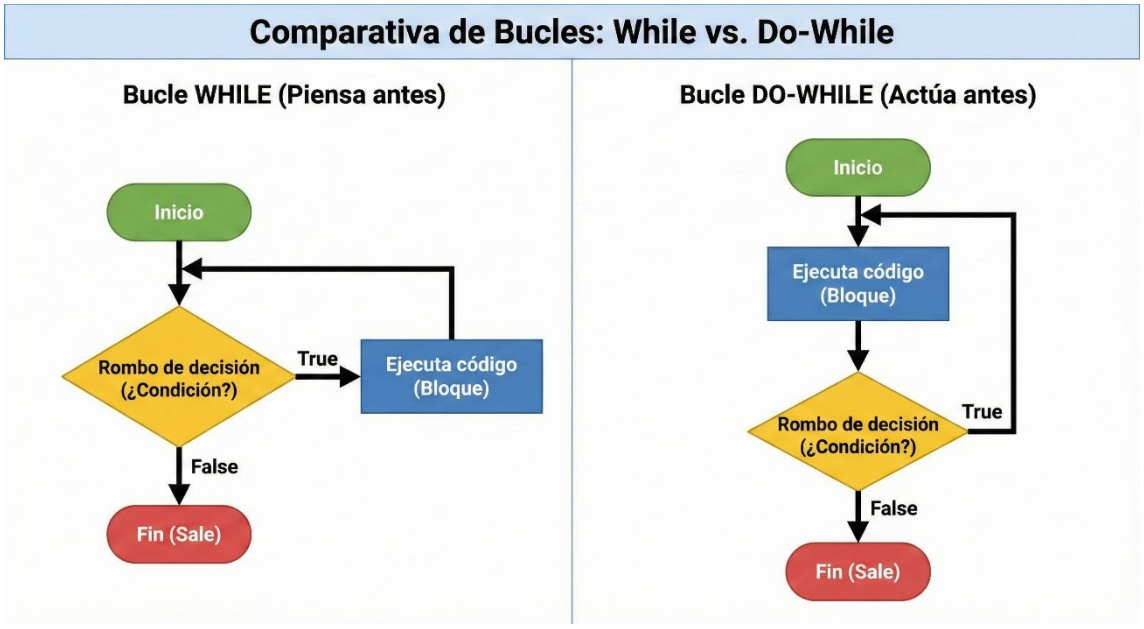
La diferencia clave es que la condición se evalúa al final. Esto garantiza que el código de dentro se ejecutará al menos una vez, pase lo que pase.

- **Uso ideal:** Menús de usuario (primero muestras el menú y luego preguntas si quiere salir).

Java
```
int numero = 10;

do {
    System.out.println("Entré al bucle aunque 10 no es menor que
5");
} while (numero < 5);

// Salida: Imprime el mensaje una vez y luego sale, porque 10 < 5
es false.
```



C. El Bucle `for` (Para)

Es el bucle más compacto y utilizado. Agrupa toda la gestión del contador en una sola línea.

- **Uso ideal:** Cuando **SÍ sabes** exactamente cuántas veces quieres repetir algo (ej: "Repetir 10 veces", "Recorrer un array de longitud 5").

Sintaxis (Las 3 partes):

for (inicialización; condición; actualización)

1. **Inicialización:** Se ejecuta una sola vez al principio (`int i = 0`).
2. **Condición:** Se evalúa antes de cada vuelta (`i < 5`). Si es false, termina.
3. **Actualización:** Se ejecuta al final de cada vuelta (`i++`).

Java
```
// Imprimir del 0 al 4
for (int i = 0; i < 5; i++) {
    System.out.println("Soy el índice: " + i);
}
```

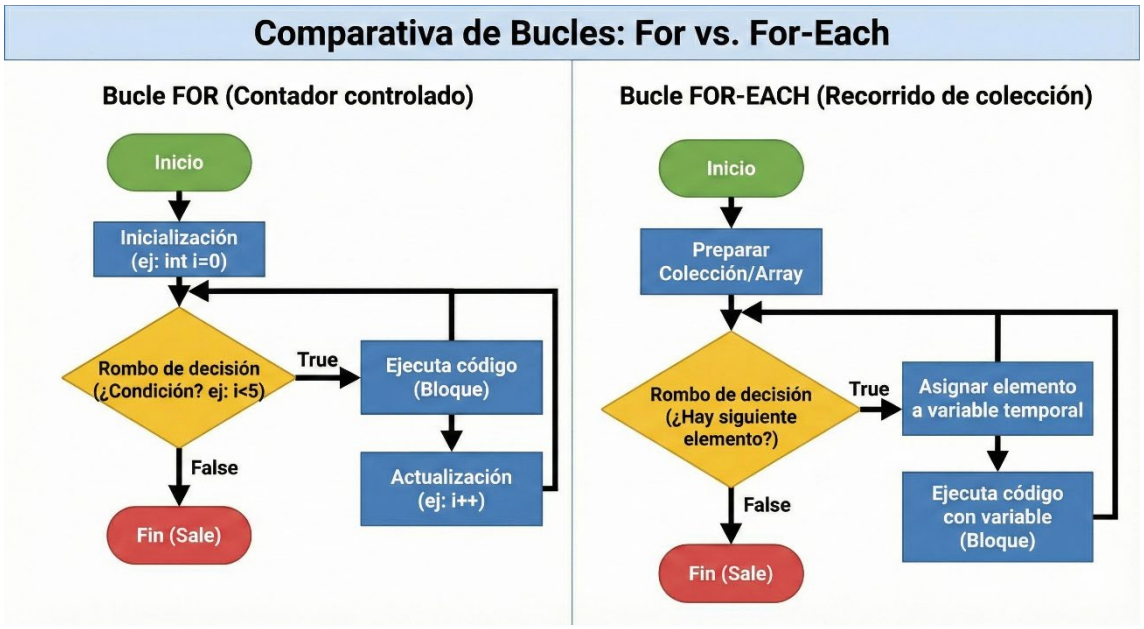
D. El Bucle `for-each` (Para cada...)

Es una versión especial y simplificada del for, diseñada exclusivamente para recorrer colecciones de datos (como Arrays o Listas) de principio a fin.

Lo veremos en profundidad cuando toquemos Arrays, pero su sintaxis es:

Java
```
// "Para cada nombre en la lista de nombres..."
for (String nombre : listaDeNombres) {
    System.out.println(nombre);
```

```
}
```



¿Cuál elijo?

Bucle	Característica	¿Cuándo usarlo?
`while`	Evalúa al principio. Puede no ejecutarse nunca.	Cuando la repetición depende de un evento externo o desconocido.
`do-while`	Evalúa al final. Se ejecuta al menos 1 vez.	Menús interactivos, validación de entradas.
`for`	Contador integrado. Compacto.	Cuando sabes el número exacto de iteraciones (N veces).

2.3 Estructuras de salto

Java dispone de tres palabras clave principales para alterar el flujo normal dentro de bucles y métodos: **`break`**, **`continue`** y **`return`**.

A. La sentencia `break` (Romper / Salir)

Ya la vimos brevemente en el `switch`. Cuando se usa dentro de un bucle (`for`, `while`, `do-while`), su función es **terminar el bucle inmediatamente**.

- **Comportamiento:** El programa "salta" fuera del bucle y continúa con la primera línea de código que haya después de la llave de cierre `}`.
- **Analogía:** Estás buscando tus llaves en 10 cajones. En el cajón 3 las encuentras. Haces un `break`. No sigues mirando en los cajones 4 al 10.

Java
```java
// Buscamos el número 3
for (int i = 1; i <= 10; i++) {
    System.out.println("Mirando el número: " + i);

    if (i == 3) {
        System.out.println("¡Encontrado!");
        break; // MATA EL BUCLE. No se imprimirán el 4, 5...
    }
}
System.out.println("Fin de la búsqueda.");
```

B. La sentencia `continue` (Continuar / Saltar vuelta)

Esta sentencia es menos drástica. No mata el bucle, solo **mata la iteración actual**.

- **Comportamiento:** Cuando Java ve un `continue`, ignora todo el código que queda por debajo *en esa vuelta* y salta directamente a evaluar la condición para la **siguiente vuelta**.
- **Analogía:** Estás pasando lista en clase. Si un alumno no ha venido (`continue`), no le pides los deberes, pasas directamente al siguiente nombre de la lista.

Java
```java
// Imprimir solo números impares del 1 al 5
for (int i = 1; i <= 5; i++) {

    // Si es par, saltamos esta vuelta
    if (i % 2 == 0) {
        continue; // Vuelve arriba al i++
    }

    // Este código no se ejecuta si entró en el if
    System.out.println("Número impar: " + i);
}
// Salida: 1, 3, 5
```

C. La sentencia `return` (Volver / Devolver)

Aunque break y continue son para bucles, return es para métodos.

Si usas un return dentro de un bucle, es la salida más potente: no solo rompe el bucle, sino que se sale del método entero donde estás.

Java
```java
public void buscarCliente(String nombre) {
    String[] clientes = {"Ana", "Juan", "Pedro"};

    for (String c : clientes) {
        if (c.equals(nombre)) {
            System.out.println("Cliente encontrado.");
            return; // FINALIZA EL MÉTODO buscarCliente() AQUÍ
MISMO
        }
    }

    System.out.println("Cliente no encontrado.");
```

```
    // Esta línea solo se ejecuta si el return de arriba nunca
ocurrió.
}
```

Comparativa de Saltos

Sentencia	Ámbito de acción	Efecto	¿Continúa el bucle?
`break`	Bucles y Switch	Salta al final del bloque `{}`.	✗ NO (Termina definitivamente)
`continue`	Solo Bucles	Salta al inicio de la siguiente vuelta.	✓ SÍ (Solo salta una iteración)
`return`	Métodos	Sale del método completo.	✗ NO (Termina bucle y función)

Más info: sobre `goto` Si vienes de otros lenguajes antiguos como C o BASIC, quizás busques la instrucción `goto`. En Java, `goto` es una palabra reservada pero no implementada. No se puede usar. Java prohíbe los saltos arbitrarios ("código espagueti") para obligar a los programadores a escribir código estructurado y limpio.

Resumen

1. Los Elementos (La Gramática)

Para que Java entienda lo que queremos, debemos respetar su estructura jerárquica estricta.

Estructura Básica

Todo programa ejecutable debe seguir este esquema de muñecas rusas:

1. **Paquete:** `package nombre;` (Carpeta contenedora).
2. **Clase:** `public class Nombre { ... }` (El archivo).
3. **Método Main:** `public static void main(String[] args) { ... }` (El punto de arranque).

Variables y Tipos de Datos

Las variables son cajas en memoria. Java es de **tipado fuerte**: la caja tiene una forma fija y no puedes meter cualquier cosa.

Categoría	Tipo	Tamaño/Valor	Uso típico
Enteros	`int`	32 bits	Contadores, IDs, matemáticas básicas.

Categoría	Tipo	Tamaño/Valor	Uso típico
	`long`	64 bits	Cifras enormes. Requiere sufijo `L`.
Decimales	`double`	64 bits	Precios, medidas. Es el estándar.
	`float`	32 bits	Ahorro de memoria. Requiere sufijo `F`.
Texto	`char`	16 bits	**Un** solo carácter: `'A'`.
	`String`	(Objeto)	Texto completo: `"Hola"`. **No es primitivo.**
Lógico	`boolean`	1 bit	`true` o `false`. Para condiciones.

Operadores y Casting

- **Aritméticos:** `+`, `-`, `*`, `/` (Cuidado: división de enteros trunca decimales), `%` (Resto).
- **Relacionales:** `==` (Igual), `!=` (Distinto), `>`, `<`. Devuelven `boolean`.
- **Lógicos:** `&&` (AND), `||` (OR), `!` (NOT).
- **Conversión (Casting):**
  - *Implícita:* De pequeño a grande (`int` -> `double`). Automática.
  - *Explícita:* De grande a pequeño (`double` -> `int`). Requiere `(int) variable`. **Riesgo de pérdida de datos.**

2. El Control de Flujo (La Lógica)

Es lo que permite que el programa tome decisiones y no sea una simple lista de instrucciones lineales.

A. Estructuras de Selección (Decisiones)

El programa evalúa una condición **booleana**.

1. **`if` / `else`:** Para condiciones generales y rangos.

   Java
   ```
   if (edad >= 18) { ... } else { ... }
   ```

2. **Operador Ternario:** El `if` en una línea.

   Java
   ```
   String estado = (edad >= 18) ? "Mayor" : "Menor";
   ```

3. **`switch`:** Para elegir entre valores discretos concretos (Menús).

Java
```
switch (opcion) {
    case 1: ... break;
    default: ...
}
```

B. Estructuras de Repetición (Bucles)

Repiten un bloque de código mientras la condición sea `true`.

Bucle	¿Cuándo se evalúa?	¿Cuándo usarlo?
`while`	Al principio.	Cuando **no sabes** cuántas veces repetirás (puede ser 0 veces).
`do-while`	Al final.	Cuando debes ejecutar **al menos 1 vez** (ej: pedir dato al usuario).
`for`	Al principio.	Cuando **sabes** el número exacto de vueltas (ej: contar hasta 10).

C. Estructuras de Salto

Modifican el comportamiento natural del bucle.

- `break`: **Rompe** el bucle totalmente (Sale fuera).
- `continue`: **Salta** solo la vuelta actual (Va a la siguiente iteración).

Sintaxis Rápida

Java
```
public class ResumenUnidad3 {
    public static void main(String[] args) {

        // 1. Declaración e Inicialización
        int contador = 0;
        final double PI = 3.1416; // Constante

        // 2. Bucle (Repetición)
        while (contador < 5) {
            contador++;

            // 3. Condicional (Selección)
            if (contador == 3) {
                continue; // Saltamos el 3
            }

            System.out.println("Numero: " + contador);
        }
        // Salida: 1, 2, 4, 5
    }
}
```